

Simulating quantitative genetic data to analyse genetic study designs

Martin Johnsson

2026-09-19

Abstract

The aim of this article is to support the use of quantitative genetic simulation for analysing genetic study designs. It presents two examples using the R-based simulation package AlphaSimR and open-source genetic analysis programs common in the field.

The first example simulates a study design to estimate a genetic correlation between two environments. This is relevant to studies of gene-by-environment effects, which can be described by the genetic correlation between a trait expressed in different environments. Simulated siblings are placed randomly into two different environments, and their trait values in the respective environments are modelled as two genetically correlated traits. Simulated data is analysed with a bivariate Bayesian animal model in MCMCglmm.

The second example simulates a genome-wide association study in an advanced intercross population started from two divergent outbred populations. This is relevant to the statistical properties of genetic mapping studies, such as linkage QTL mapping and genome-wide association studies. Simulated individuals are randomly crossed for a number of generations, then expanded to a mapping population. Simulated genotype and phenotype data is analysed with a linear mixed model in GEMMA.

The examples meant to serve as a starting point for creating custom simulations of population structures and data analyses that reflect the study designs that the reader is considering. Step-by-step instructions as well as full source code is provided.

Introduction

The aim of this article is to support the use of quantitative genetic simulation to analyse genetic study designs. Study designs like these are ubiquitous in quantitative, evolutionary, animal and plant genetics. We estimate genetic parameters, predict selection responses and identify loci associated with traits. Simulation would help understand the properties of quantitative genetic studies, help build intuition, and support teaching and learning of genetic data analysis.

Therefore, this article presents two quantitative genetic simulation examples with AlphaSimR (Gaynor, Gorjanc, and Hickey 2021). AlphaSimR is a flexible, scriptable simulation package running in the R statistical environment (R Core Team 2022). The original purpose is breeding program simulation, and the package therefore allows scripting complex breeding structures (including simulating polyploidy and biotechnological methods used in plant breeding). There are multiple other options for genetic simulations of quantitative traits (Sargolzaei and Schenkel 2009; Howard et al. 2017; Pook, Schlather, and Simianer 2020; Pook et al. 2021), and the logic and workflow of these examples should apply regardless of what simulation software you prefer.

The simulations are built up in two steps. The genomes of the founder generations, the first generation of simulated individuals, are generated by a coalescent simulator (MaCS or msprime), which simulates sequences under neutral evolution (Chen, Marjoram, and Wall 2009; Baumdicker et al. 2022). A subset of the segregating sites from the founder genomes are picked to be causative variants, and used to simulate genetic and phenotypic values for traits. Then, AlphaSimR performs individual-based forward-in-time simulations, allowing for reproduction, selection and so on. In these examples, we then take the data from simulation

forward to genetic data analyses, and compare the inferences from simulated data to true parameter values. There will be two case studies: a study design to estimate genetic correlation between two environments using an animal model and a genome-wide association study in a structured population.

The first example simulates a study design to estimate a genetic correlation between two environments. This is relevant to readers aiming to study gene-by-environment effects, which can be described by the genetic correlation between a trait expressed in different environments. It simulates a controlled experiment where siblings are randomly allocated to two different environments and measured there. The genetic correlation is estimated with a bivariate Bayesian mixed model with the MCMCglmm package (Hadfield 2010). The same simulation can be modified to reflect other quantitative genetic studies for estimating the heritability or the genetic correlation between distinct traits in the same environment. The simulation is meant to serve as a starting point for creating custom population structures that reflect the study designs that the reader is considering.

The second example simulates a genome-wide association study in an advanced intercross population started from two divergent and outbred but small populations. In order to have more realistic linkage disequilibrium structure within the founder chromosomes, the founder population is simulated with hybrid coalescent and Wright–Fisher simulation (Nelson et al. 2020). The genome-wide association study is performed with linear mixed models controlling relatedness with a genome relationship matrix with the GEMMA software (Zhou and Stephens 2012) (supported by Plink for preparing the data files (Chang et al. 2015)). This is relevant to readers aiming to understand the statistical properties genetic mapping studies, such as linkage QTL mapping and genome-wide association studies. As in the first example, this particular population structure is only meant as a starting point for the reader to tailor a simulation to their own study. For a simulation tool that explicitly addresses common designs with inbred strains used in rodent genetics, see (Gregory R. Keele et al. 2019; Gregory R. Keele 2022). The second example is more involved in terms of customising the breeding structure and genetic architecture.

Methods

Generic protocol

Both the case studies will follow the same generic protocol for building a simulation to analyse a genetic study design:

1. Define the founder population history.
2. Define simulation parameters.
3. Define genetic architecture.
4. Build the breeding structure.
5. Prepare simulated data.
6. Run genetic analysis.
7. Summarise results.
8. Repeat.
9. Visualise outcomes.

Example 1: Simulating a structured population to evaluate quantitative genetic estimates

Quantitative genetic estimates, such as (additive) genetic variances, heritability coefficients and genetic correlations are key to evolutionary and applied quantitative genetics, and they typically require large sample sizes for accurate estimation.

This case study analyses a design for estimating the genetic correlation between a trait in two environments. Closely related individuals, in this case siblings from random crosses within a population, are randomised

into two different environments. Traits are recorded once for each individual. This design could be used to quantify gene-by-environment effects between two different rearing environments.

1. Define the founder population history We simulate founder chromosomes for 240 individuals, drawn from a population with the AlphaSimR default “GENERIC” population history, which is supposed to roughly reflect the history a domestic animal population with gradual decreases in effective population size to a small contemporary effective population size.

```
founders <- runMacs(nInd = 240,  
                  nChr = 10)
```

2. Define simulation parameters AlphaSimR stores parameters for the simulation in a `simParam` object that needs to be passed to various functions later.

2.1 Create the new simulation. This initialises the simulation parameters object.

```
simparam <- SimParam$new(founders)
```

2.2 Switch on sexes. This tells AlphaSimR to systematically make half the individuals male and female.

```
simparam$setSexes("yes_sys")
```

2.3 Switch on pedigree tracking. Because we are performing a pedigree-based quantitative genetic study, the pedigree of the simulated individuals will be part of the simulated data.

```
simparam$setTrackPed(TRUE)
```

3. Define genetic architecture We will represent a trait expressed in two different environments as two correlated traits. The genetic correlation between the two environments can be thought of as reflection of gene-by-environment interaction.

3.1 Define two correlated traits. The relationship between the traits is parameterised by the correlation between additive genetic effects. Here, we set the correlation between additive effects to 0.5, the means to 0, and variances to 1.

```
corA <- 0.5  
simparam$addTraitA(nQtlPerChr = 10,  
                  mean = c(0, 0),  
                  var = c(1, 1),  
                  corA = matrix(c(1, corA, corA, 1), ncol = 2))
```

3.2 Define the environmental variance. Here, we set the heritability of each trait to 0.3.

```
simparam$setVarE(h2 = c(0.3, 0.3))
```

4. Build the breeding structure Because populations are represented as R objects, we can be very flexible with the simulation of the breeding structure, allowing for complex mating designs. The breeding structure for this simulation, however, simply consists of one generation for random mating.

4.1 Create the population object. This creates a population object for the founders from the founder chromosomes and simulation parameters.

```
pop <- newPop(founders,  
             simParam = simparam)
```

4.2 Randomly cross founders. We generate 1200 offspring, split into 120 families.

```
offspring <- randCross(pop = pop,  
                     nProgeny = 10,
```

```
nCrosses = 120,
simParam = simparam)
```

5. Prepare simulated data In this step, we prepare the datasets from the simulation that will be used for quantitative genetic estimation. In this case, it consists of trait values and pedigree.

5.1 Divide individuals between environments. Here, we place half of the individuals (and half from each family) in each environment, by generating a categorical environmental covariate.

```
environment <- rep(c(1, 2), 600)
```

5.2 Extract the trait values. AlphaSimR will generate trait values for both our correlated traits. Therefore, we use the environmental covariate to keep only the appropriate trait value for each individual (that corresponding to its environment), and setting the other to missing (NA).

```
pheno_data <- data.frame(animal = offspring@id,
                        trait1 = scale(offspring@pheno[,1]),
                        trait2 = scale(offspring@pheno[,2]),
                        environment = environment,
                        stringsAsFactors = FALSE)
pheno_data$trait1[environment == 2] <- NA
pheno_data$trait2[environment == 1] <- NA
```

5.3 Extract the pedigree. We use the pedigree recorded by AlphaSimR, with two minor modifications: We add an identifier for each individual called `animal`, and recode the missing values (for founder individuals, who have no known parents) from 0 to missing values (NA). These changes are specific to the quantitative genetic package (MCMCglmm) that we will use.

```
ped <- data.frame(animal = c(pop@id, offspring@id),
                 simparam$pedigree[, -3])
ped$mother[ped$mother == 0] <- NA
ped$father[ped$father == 0] <- NA
```

6. Run genetic analysis In this step, we use the Bayesian linear mixed model package MCMCglmm to fit a bivariate animal model to estimate the genetic correlation between the two environments.

6.1 Set up the prior. MCMCglmm uses inverse-Wishart priors for variance components, here assuming that the variance is split equally between genetic and residual variance.

```
prior <- list(G = list(G1 = list(V = diag(2)/2, nu = 2)),
             R = list(V = diag(2)/2, nu = 2))
```

6.2 Fit the linear mixed model.

```
model <- MCMCglmm(cbind(trait1, trait2) ~ 1,
                 random = ~us(trait):animal,
                 rcov = ~us(trait):units,
                 family = c("gaussian", "gaussian"),
                 pedigree = ped,
                 data = pheno_data,
                 prior = prior)
```

7. Summarise results After analysing the simulated data, the goal is to produce a summary that can be stored as a row of a data frame. This will simplify matters when we run the simulation in many replicates.

7.1 Estimate heritability for the two traits. The heritability, namely the ratio $h^2 = \frac{V_a}{V_a + V_e}$, is formed from posterior samples.

```
h2_trait1 <- model$VCV[, "traittrait1:traittrait1.animal"] /
  (model$VCV[, "traittrait1:traittrait1.units"] +
    model$VCV[, "traittrait1:traittrait1.animal"])

h2_trait2 <- model$VCV[, "traittrait2:traittrait2.animal"] /
  (model$VCV[, "traittrait2:traittrait2.units"] +
    model$VCV[, "traittrait2:traittrait2.animal"])
```

7.2 Estimate genetic correlation between them. Similarly, we form the genetic correlation $r_g = \frac{Cov_{1,2}}{\sqrt{V_{a,1} \cdot V_{a,2}}}$ from posterior samples.

```
rg <- model$VCV[, "traittrait1:traittrait2.animal"] /
  sqrt(model$VCV[, "traittrait1:traittrait1.animal"] *
    model$VCV[, "traittrait2:traittrait2.animal"])
```

7.3 Summarise in a data frame. We put these estimates, with their posterior mean and credible interval, into a data frame together with the true correlation between additive effects used to generate the data and the true genetic correlation that resulted in from it.

```
data.frame(cor_a = corA,
  cor_true = cor(pop@gv[,1], pop@gv[,2]),
  rg_est = mean(rg),
  rg_lower = HPDinterval(rg)[1],
  rg_upper = HPDinterval(rg)[2],
  h2_trait1 = mean(h2_trait1),
  h2_trait1_lower = HPDinterval(h2_trait1)[1],
  h2_trait1_upper = HPDinterval(h2_trait1)[2],
  h2_trait2 = mean(h2_trait2),
  h2_trait2_lower = HPDinterval(h2_trait2)[1],
  h2_trait2_upper = HPDinterval(h2_trait2)[2])

##   cor_a cor_true  rg_est  rg_lower rg_upper h2_trait1 h2_trait1_lower
## 1    0.5 0.558032 0.478323 0.1645086 0.742696 0.3145779    0.1886627
##   h2_trait1_upper h2_trait2 h2_trait2_lower h2_trait2_upper
## 1           0.4639887 0.2600187          0.1205683          0.402776
```

8. Replicate There is a limit to what can be learned from one simulation replicate. Therefore, we will replicate the simulation many times while sweeping over one of the parameters (the correlation between the additive effects in the two environments).

8.1 Create a grid of parameters. We create a vector of correlation values to explore, going from -0.7 to 0.7 in increments of 0.1, but ignoring the values closest to zero.

```
cor_as <- rep(seq(from = -0.7, to = 0.7, by = 0.1), each = 50)
cor_as <- cor_as[!(cor_as >= -0.11 & cor_as < 0.11)]
```

8.2 Replicate the simulation. We run the entire simulation over this grid of parameter values. This piece of code relies on having the previous steps of the simulation encapsulated in functions. These can be viewed in the underlying source code (https://github.com/mrtnj/introduction_power_simulation).

```
results <- map_dfr(cor_as,
  function(x) {
    simulation <- make_simulation(x)
    data <- get_data(simulation$founders,
```

```

simulation$offspring,
simulation$simparam)

model <- estimate_mcmcglmm(data$pheno_data,
                           data$ped)

get_results(model,
            simulation$offspring,
            x,
            simulation$true_cor_g)
})

```

9. Visualise outcomes NOTE: The whole visualisation code for the representative results is available at https://github.com/mrtnj/introduction_power_simulation, and contains more plots. Here we give just one example.

9.1 Label simulation runs based on whether the credible interval excludes zero or not. This code declares an estimate of the genetic correlation “significant” if the credible interval excludes zero.

```

results$significant <- ifelse(!(results$rg_lower <= 0 &
                               results$rg_upper > 0),
                             "significant",
                             "not significant")

```

9.2 Graph the estimates (posterior mean and credible interval) with true values using ggplot2 (Wickham 2016). .

```

plot_all_significant <- ggplot() +
  geom_pointrange(aes(x = cor_true,
                     y = rg_est,
                     ymin = rg_lower,
                     ymax = rg_upper,
                     colour = significant),
                 size = 0.25,
                 data = results) +
  geom_hline(yintercept = 0,
             linetype = 2) +
  ylim(-1, 1) +
  scale_colour_manual(values = c("grey", "black")) +
  theme_bw() +
  theme(panel.grid = element_blank()) +
  xlab("True genetic correlation") +
  ylab("Estimated genetic correlation") +
  ggtitle("Estimates and credible intervals")

```

Example 2: Simulating genome-wide association studies in an advanced intercross

1. Define the founder population history We simulate founder chromosomes for 240 individuals from AlphaSimR’s default “GENERIC” population history, with a population split 100 generations ago.

```

founders_gwas <- runMacs(nInd = 240,
                        nChr = 10,
                        split = 100)

```

2. Define simulation parameters 2.1 Create a new simulation.

```
simparam_gwas <- SimParam$new(founders_gwas)
```

2.2 Switch on sexes.

```
simparam_gwas$setSexes("yes_sys")
```

2.3 Add a simulated SNP chip. Because we are testing a genetic mapping design, we need to simulate genotyping. We can do this by defining a SNP chip, here with 1000 randomly selected markers per chromosome.

```
simparam_gwas$addSnpChip(nSnpPerChr = 1000)
```

3. Define genetic architecture 3.1 Select causative variants with a frequency threshold. We pick one causative variant per chromosome, randomly from the variants that have higher than 1% allele frequency. This is to avoid variants with very low frequency.

```
n_chr <- founders_gwas@nChr
qtl_ix <- numeric(n_chr)
p <- numeric(n_chr)

threshold <- 0.01

for (chr in 1:n_chr) {
  geno <- pullSegSiteGeno(founders_gwas,
                          chr = chr,
                          simParam = simparam)

  f <- colSums(geno)/2/nrow(geno)

  candidate_qtl <- which(f > threshold & f < 1 - threshold)

  qtl_ix[chr] <- sample(candidate_qtl, 1)
  p[chr] <- f[qtl_ix[chr]]
}
```

3.2 Calculate the additive and dominance coefficients at each locus. Given a set desired genetic variance for each locus ($V'_a = 0.1$ and a desired dominance degree $\delta = 0.2$, we can calculate the additive and dominance coefficients based on the allele frequency p (see Discussion).

```
Va <- 0.1
delta <- 0.2
a <- sqrt(Va / (2 * p * (1 - p) * (1 + delta * (1 - p - p))^2))
d <- a * delta
```

3.3 Add the trait to the simulation parameters. Here we used the *manAddTrait* function to manually add our trait, and set the environmental variance to 3 for a heritability of $h^2 \approx 0.33$.

```
trait <- new("TraitAD",
            intercept = 0,
            nLoci = as.integer(n_chr),
            lociPerChr = rep(1L, n_chr),
            lociLoc = as.integer(qtl_ix),
            addEff = a,
            domEff = d)

simparam_gwas$manAddTrait(lociMap = trait,
                          varE = 3)
```

4. Build the breeding structure 4.1 Create the population object.

```
pop <- newPop(founders_gwas,  
             simParam = simparam_gwas)
```

4.2 Split founders from the two populations. Half of the founder individuals are derived from each population, i.e., 120 each.

```
pop_line1 <- pop[1:120]  
pop_line2 <- pop[121:240]
```

4.2 Randomly cross individuals for multiple generations. We simulate a population that is maintained at a population size of 240 individuals for 9 generations after the first cross.

```
generations <- vector(mode = "list",  
                      length = 10)  
  
generations[[1]] <- randCross2(females = pop_line1,  
                               males = pop_line2,  
                               nProgeny = 2,  
                               nCrosses = 120,  
                               simParam = simparam_gwas)  
  
for (gen in 2:10) {  
  generations[[gen]] <- randCross(pop = generations[[gen - 1]],  
                                  nProgeny = 2,  
                                  nCrosses = 120,  
                                  simParam = simparam_gwas)  
}
```

4.3 Expand to generate the mapping population. We expand the final generation to a mapping population of 1200.

```
offspring <- randCross(pop = generations[[10]],  
                      nProgeny = 10,  
                      nCrosses = 120,  
                      simParam = simparam_gwas)
```

5. Prepare simulated data 2.1 Export genotypes and phenotypes in Plink format. Because GEMMA, the genome-wide association tool we will use, uses binary Plink format, we can use the built in exportation function, and later convert the Plink text format to binary format.

```
dir.create("simulations")  
  
writePlink(offspring,  
           "simulations/intercross_data",  
           simParam = simparam_gwas)
```

NOTE: To access the SNP genotypes and phenotypes directly, one can use `pullSnpGeno` and `offspring@pheno` respectively. This might be needed to create other file formats for other genome-wide association or linkage mapping tools.

2.2 Export metadata about causative variants. To be able to evaluate the results, we need to know where the causative variants are, and potentially their effects. Therefore, we extract and calculate a number of statistics about the causative variants (positions, effects, variance explained and frequencies in the founder and mapping populations).

```

true_qtl <- getQtlMap(simParam = simparam_gwas)
true_qtl$a <- a
true_qtl$d <- d
true_qtl$founder_p <- p

qtl_geno <- pullQtlGeno(offspring,
                        simParam = simparam_gwas)
true_qtl$mapping_p <- colSums(qtl_geno)/2/nrow(qtl_geno)
true_qtl$mapping_Va <- 2 * true_qtl$mapping_p * (1 - true_qtl$mapping_p) *
  (a + d * (1 - 2 * true_qtl$mapping_p))^2

offspring_varP <- as.numeric(varP(offspring))
true_qtl$mapping_var_explained <- true_qtl$mapping_Va/offspring_varP

write.table(true_qtl,
            file = "simulations/true_qtl.txt",
            quote = FALSE,
            row.names = FALSE,
            sep = "\t")

```

6. Run genetic analysis NOTE: The code blocks in this section are not R code but shell code, to be executed by a shell interpreter such as `bash` (default in several Linux distributions) or `zsh` (default on Mac OS). It is possible to run such code from inside an R script, but the resulting script may not be any clearer.

6.1 Convert Plink text format to binary with Plink. This assumes that you have Plink installed.

```

set -eu

plink --file simulations/intercross_data \
      --make-bed \
      --out simulations/intercross_data

```

6.2 Run genome-wide association with GEMMA. This assumes that you have GEMMA installed. The first step estimates the genomic relationship matrix from genotypes, while the second step uses the genomic relationship matrix, genotypes and phenotypes to fit the linear mixed model for all the markers.

```

set -eu

cd simulations

gemma -bfile intercross_data \
      -gk 1 \
      -o intercross_data

gemma -bfile intercross_data \
      -k output/intercross_data.cXX.txt \
      -lmm 4 \
      -o intercross_data

```

7. Summarise results 7.1 Read results and metadata.

We read in the summary statistics from GEMMA as well as the metadata we saved about the simulated causative variants.

```

results <- read_tsv("simulations/output/intercross_data.assoc.txt")
true <- read_tsv("simulations/true_qtl.txt")

```

7.2 A function to summarise a genome scan.

In order to summarise the results of a genome scan of a simulated dataset, we find the distance between any significant markers (at a given `threshold`) to the true causative variant, and if it is within a given distance (`interval`), we declare the variant detected. All other significant markers are counted as false positives.

```
summarise_scan <- function(results,
                           true,
                           threshold,
                           interval) {

  true$pos_bp <- true$pos * 1e8

  results_split <- split(results, results$chr)
  true_split <- split(true, true$chr)

  n_chr <- length(results_split)
  detected <- logical(n_chr)
  false_positives <- numeric(n_chr)

  for (chr_ix in 1:n_chr) {
    results_on_chr <- results_split[[chr_ix]]
    sign <- results_on_chr[results_on_chr$p_wald < threshold,]
    distance <- abs(sign$ps - true_split[[chr_ix]]$pos_bp)
    if (any(distance < interval/2)) {
      detected[chr_ix] <- TRUE
    }
    false_positives[chr_ix] <- sum(distance > interval/2)
  }

  true$detected <- detected
  true$false_positives <- false_positives

  true

}

summarise_scan(results, true, 1e-5, 1e6)[,c("chr", "pos", "a", "d",
                                             "mapping_var_explained",
                                             "false_positives")]
```

```
## # A tibble: 10 x 6
##   chr    pos    a    d mapping_var_explained false_positives
##   <dbl> <dbl> <dbl> <dbl>                <dbl>          <dbl>
## 1     1  1 0.0601 0.660 0.132                0.0275          1
## 2     2  2 0.306  0.496 0.0992                0.0227          0
## 3     3  3 0.874  0.440 0.0880                0.0233          1
## 4     4  4 0.621  0.440 0.0880                0.0228          1
## 5     5  5 0.0492 0.786 0.157                0.0406          0
## 6     6  6 0.150  0.709 0.142                0.0327          2
## 7     7  7 0.609  0.439 0.0878                0.0239          0
## 8     8  8 0.917  0.567 0.113                0.0249          0
## 9     9  9 0.155  0.454 0.0909                0.0225          0
## 10    10 0.892  0.937 0.187                0.0280          0
```

8. Replicate Because this simulation involves saving data files and running an external analysis program on them, the easiest way to replicate the simulation is to wrap each part in a loop.

NOTE: The whole code for replicating the simulation code is available at https://github.com/mrtnj/introduction_power_simulation.

8.1 Replicate the simulation of data by wrapping the R code in a `for` loop.

```
for (rep in 1:20) {  
  
  ## Run simulation as above  
  
}
```

8.2 Replicate the data analysis by wrapping the script in `for` loop (the syntax differences are due to the differences between R and the shell).

```
set -eu  
  
for REP in {1..20}; do  
  
  ## Run Plink or GEMMA as above  
  
done
```

9. Visualise outcomes 9.1 Plots of one replicate dataset.

Here, we use the qqman R package (Turner 2018) to draw the typical Manhattan plot (of negative logarithm of p-values against genomic position) and quantile–quantile plot from one replicate of simulated data.

```
par(mfcol = c(1, 2))  
manhattan(results, chr = "chr", bp = "ps", p = "p_wald", snp = "rs")  
qq(results$p_wald)
```

Results

Example 1:

Figure 1 shows typical outcomes of example 1: point estimates, credible intervals and an empirical estimate of power of a bivariate animal model applied to simulated data. The study simulated in example 1 has good power to exclude a zero correlation only when the magnitude of the genetic correlation is large (> 0.6). However, if the genetic correlation between environments is low, at least the study should give a clue that the genetic correlation is small, even if will struggle to estimate if the correlation is low positive to low negative.

Thus, the uncertainty at this sample size is big, with an average length of a credible interval for the correlation of about 0.6 units wide. However, the ability to tell what sign the correlation is (unless it is close to zero) and the ability to exclude a strong positive correlation (which would signify little to no difference between the environments) are quite good.

On the other hand, the average length of the credible intervals for the heritability estimates are shorter, around 0.28 units. This illustrates the old observation of Robertson (1959) that a genetic correlation is much harder to estimate and needs a much larger sample size than a heritability.

Example 2

Figure 2 shows typical outcomes of one replicate of the simulation in example 2: a Manhattan plot and a quantile–quantile plot of the negative logarithm of p-values from the genome-wide association study on simulated data. Over 20 replicates, at a threshold of $p < 5 \cdot 10^{-8}$, the power to detect a quantitative locus was estimated to be 0.085 with on average 0.15 false positive associations per genome scan. If the threshold

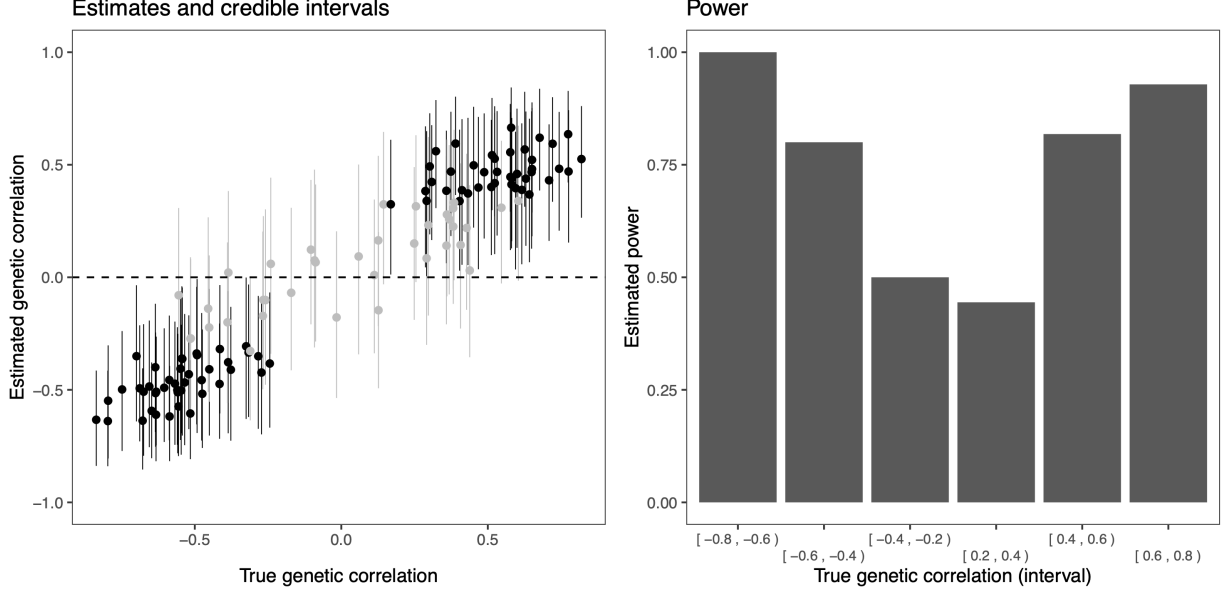


Figure 1: Figure 1. The left panel shows point estimates for the genetic correlation with credible intervals, with true genetic correlation on the horizontal axis. The intervals are coloured black if they exclude zero, and grey if they do not. The right panel shows the estimated power to detect a non-zero correlation (fraction of intervals that exclude zero) in bins of the true genetic correlation.

was relaxed to $p < 10^{-5}$, the power was 0.295 with 1.65 false positives. Finally, if the threshold was lowered to the extremely permissive $p < 10^{-3}$, the power was 0.63 and the average number of false positives 16.5. The simulated variants explain around 2.5% of the phenotypic variation in the mapping population.

Discussion

This paper has presented two example simulations as starting points for analysing common genetic study designs. The discussion will point to some limitations and and possible extensions.

The population history used to simulate the founder genomes is likely not very realistic. The population history affects the number of segregating sites, their frequency spectrum and the correlation between them (linkage disequilibrium). Coalescent simulation is known to have problems with unrealistic linkage disequilibrium when the effective population size is small compared to the sample size (Nelson et al. 2020), as is the case for the founder generation in these simulations. Instead of coalescent simulation, pre-existing founder genomes could be imported with **newMapPop**. These could either be derived from (statistically phased) real genome sequences or generated with another simulator, such as a forward-simulator that allows for more control of the population history, mutation and selection.

The way genetic architecture is defined in the two examples differ, because in the genetic mapping study (example 2), we needed more control of the effect sizes. By default, AlphaSimR draws the causative variants randomly from the chromosome, and the effects randomly from a normal (or optionally gamma) distribution, and finally tunes the effects to get the desired genetic variance (see Gaynor, Gorjanc, and Hickey (2021) for details). In example 2, we instead drew random causative variants subject to an allele frequency constraint and then calculated the additive genetic coefficient needed to give a particular additive genetic variance per locus V'_a , given a particular dominance degree δ , assuming that the dominance coefficient is a fraction of the additive genetic coefficient $d = \delta a$. The formula is derived from the formula for additive genetic variance under random mating and linkage equilibrium $V_a = 2p(1-p)[a + d(1-2p)]^2$ (Falconer and Mackay 1996). This formula can be rearranged to:

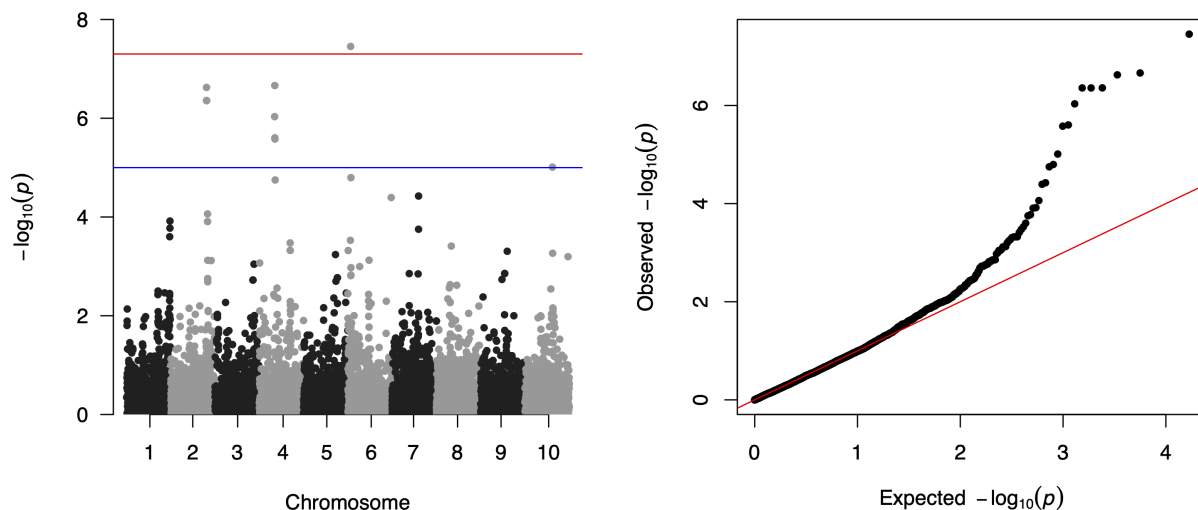


Figure 2: Figure 2. Manhattan plot and quantile–quantile plot of one replicate of a genome-wide association study on a simulated dataset.

$$a = \sqrt{\frac{V'_a}{2p(1-p)[1 + \delta(1-2p)]^2}}$$

These simulations have many parameters that could be varied, both in terms of the genetic architecture (number, frequency and effects of causative variants) and the inference methods (e.g., the priors used in MCMCglmm or the specification of the GWAS model). This is another reason why it is useful to have access to flexible simulations that can be easily modified and extended; to study the properties of your particular study, you are likely to need a simulation tailored to your study design.

If we were to draw a general conclusion from these simulations, one lesson might be that quantitative genetic inference is hard. Whether we aim to estimate quantitative genetic parameters like genetic correlations or identify loci that contribute to quantitative traits, very large samples are needed if the methods are to estimate parameters precisely or identify small-effect loci. Hopefully, analysis of study designs by simulation can help clarify what aspects of a study are informative about particular parameters, and which inference tasks are too difficult for the study design at hand.

References

- Baumdicker, Franz, Gertjan Bisschop, Daniel Goldstein, Graham Gower, Aaron P Ragsdale, Georgia Tsambos, Sha Zhu, et al. 2022. “Efficient Ancestry and Mutation Simulation with Msprime 1.0.” *Genetics* 220 (3): iyab229. <https://doi.org/10.1093/genetics/iyab229>.
- Chang, Christopher C, Carson C Chow, Laurent CAM Tellier, Shashaank Vattikuti, Shaun M Purcell, and James J Lee. 2015. “Second-Generation PLINK: Rising to the Challenge of Larger and Richer Datasets.” *GigaScience* 4 (1): s13742-015-0047-8. <https://doi.org/10.1186/s13742-015-0047-8>.
- Chen, Gary K., Paul Marjoram, and Jeffrey D. Wall. 2009. “Fast and Flexible Simulation of DNA Sequence Data.” *Genome Research* 19 (1): 136–42. <https://doi.org/10.1101/gr.083634.108>.
- Falconer, D. S., and Trudy F. C. Mackay. 1996. *Introduction to Quantitative Genetics*. Subsequent edition. Harlow: Benjamin-Cummings Pub Co.
- Gaynor, R Chris, Gregor Gorjanc, and John M Hickey. 2021. “AlphaSimR: An R Package for Breeding Program Simulations.” *G3 Genes/Genomes/Genetics* 11 (jkaa017). <https://doi.org/10.1093/g3journal/jk>

aa017.

- Hadfield, Jarrod D. 2010. "MCMC Methods for Multi-Response Generalized Linear Mixed Models: The MCMCglmm R Package." *Journal of Statistical Software* 33 (2): 1–22.
- Howard, J. T., F. Tiezzi, J. E. Pryce, and C. Maltecca. 2017. "Geno-Diver: A Combined Coalescence and Forward-in-Time Simulator for Populations Undergoing Selection for Complex Traits." *Journal of Animal Breeding and Genetics* 134 (6): 553–63. <https://doi.org/10.1111/jbg.12277>.
- Keele, Gregory R. 2022. "Which Mouse Multiparental Population Is Right for Your Study? The Collaborative Cross Inbred Strains, Their F1 Hybrids, or the Diversity Outbred Population." *bioRxiv*. <https://doi.org/10.1101/2022.08.26.505416>.
- Keele, Gregory R, Wesley L Crouse, Samir N P Kelada, and William Valdar. 2019. "Determinants of QTL Mapping Power in the Realized Collaborative Cross." *G3 Genes/Genomes/Genetics* 9 (5): 1707–27. <https://doi.org/10.1534/g3.119.400194>.
- Nelson, Dominic, Jerome Kelleher, Aaron P. Ragsdale, Claudia Moreau, Gil McVean, and Simon Gravel. 2020. "Accounting for Long-Range Correlations in Genome-Wide Simulations of Large Cohorts." *PLOS Genetics* 16 (5): e1008619. <https://doi.org/10.1371/journal.pgen.1008619>.
- Pook, Torsten, Lisa Büttgen, Amudha Ganesan, Ngoc-Thuy Ha, and Henner Simianer. 2021. "MoBPSweb: A Web-Based Framework to Simulate and Compare Breeding Programs." *G3 Genes/Genomes/Genetics* 11 (2): jkab023. <https://doi.org/10.1093/g3journal/jkab023>.
- Pook, Torsten, Martin Schlather, and Henner Simianer. 2020. "MoBPS-Modular Breeding Program Simulator." *G3: Genes, Genomes, Genetics* 10 (6): 1915–18.
- R Core Team. 2022. *R: A Language and Environment for Statistical Computing*. Vienna, Austria: R Foundation for Statistical Computing. <https://www.R-project.org>.
- Robertson, Alan. 1959. "The Sampling Variance of the Genetic Correlation Coefficient." *Biometrics* 15 (3): 469–85. <https://doi.org/10.2307/2527750>.
- Sargolzaei, Mehdi, and Flavio S Schenkel. 2009. "QMSim: A Large-Scale Genome Simulator for Livestock." *Bioinformatics* 25 (5): 680–81. <https://doi.org/10.1093/bioinformatics/btp045>.
- Turner, Stephen. 2018. "Qqman: An r Package for Visualizing GWAS Results Using q-q and Manhattan Plots." *The Journal of Open Source Software*. <https://doi.org/10.21105/joss.00731>.
- Wickham, Hadley. 2016. *Ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. <https://ggplot2.tidyverse.org>.
- Zhou, Xiang, and Matthew Stephens. 2012. "Genome-Wide Efficient Mixed-Model Analysis for Association Studies." *Nature Genetics* 44 (7): 821–24. <https://doi.org/10.1038/ng.2310>.